

CS 3563: Introduction to DBMS II

Homework 2: Storage Management and Indexing

Name: Pathri Vidya Praveen
Roll Number: CS24BTECH11047

Question 1

Given the eviction score formula:

$$\text{EvictionScore}(i) = \frac{\text{AccessRate}(i) \times \text{ReadCost}(i)}{\text{Size}(i)}$$

We first calculate the eviction score for all objects currently in the buffer pool:

Object	Size (MB)	Read Cost (ms)	Refs (10s)	Calculation	Score
A	4	50	20	$\frac{20 \times 50}{4}$	250
B	2	200	5	$\frac{5 \times 200}{2}$	500
C	3	30	2	$\frac{2 \times 30}{3}$	20
D	1	100	8	$\frac{8 \times 100}{1}$	800

1.

A new object E of size 3 MB needs to be brought into the buffer pool. The buffer manager prefers to evict objects with the lowest eviction scores.

Looking at the calculated scores:

- Object C has the lowest score (20).
- Object C has a size of 3 MB.

Since evicting Object C frees up exactly 3 MB which is sufficient for Object E and C has the lowest eviction score (minimizing the cost per unit space lost), it is the optimal choice.

Object C should be evicted.

2.

We must compare evicting Object A versus evicting the combination of Objects C and D. The preference is determined by the total expected cost (value lost) per unit of buffer space freed.

Option A: Evict Object A

- Total Space Freed: 4 MB
- Total Value Lost: $\text{AccessRate} \times \text{ReadCost} = 20 \times 50 = 1000$
- Eviction Score:

$$\text{Score}_A = \frac{1000}{4} = 250$$

Option B: Evict Objects C and D together

- Total Space Freed: $\text{Size}(C) + \text{Size}(D) = 3 + 1 = 4$ MB
- Total Value Lost (C): $2 \times 30 = 60$
- Total Value Lost (D): $8 \times 100 = 800$
- Combined Value Lost: $60 + 800 = 860$
- Eviction Score:

$$\text{Score}_{C+D} = \frac{860}{4} = 215$$

Conclusion Since the combined eviction score for C and D (215) is lower than the eviction score for A (250), evicting C and D incurs a lower expected cost per unit of space.

It is preferable to evict (b) objects C and D together.

Question 2

1. Stale FSM Entries at Time T_1

First, we define the Free Space Map (FSM) thresholds based on the block capacity of 100 records:

- 00 (0–30%): 0–30 records
- 01 (30–60%): 31–60 records
- 10 (60–90%): 61–90 records
- 11 (90–100%): 91–100 records

We compare the stored FSM bits (based on T_0 occupancy) with the actual occupancy at T_1 after the updates.

Block	Records (T_0)	FSM (T_0)	Update	Records (T_1)	Correct FSM (T_1)	Stale?
1	62	10	+5	67	10	No
2	85	10	+10	95	11	Yes
3	20	00	+60	80	10	Yes
4	45	01	-10	35	01	No
5	75	10	+20	95	11	Yes
6	95	11	–	95	11	No

The blocks with stale FSM entries at time T_1 are Block 2, Block 3 and Block 5.

2. Final FSM State After Insertion

At time T_1 , we attempt to insert 30 records. The system checks FSM entries in order to find blocks that guarantee enough space for the current batch.

Insertion Process:

1. Block 1 (FSM 10): Max occupancy 90 $\rightarrow$ Min free space 10. ($10 < 30$). Skipped
2. Block 2 (FSM 10): Max occupancy 90 $\rightarrow$ Min free space 10. ($10 < 30$). Skipped
3. Block 3 (FSM 00): Max occupancy 30 $\rightarrow$ Min free space 70. ($70 \geq 30$).
 - The system attempts to insert all 30 records here.
 - Actual Space: Block 3 has 80 records. Free space = $100 - 80 = 20$.
 - Action: Insert 20 records. Block 3 becomes full.
 - Remaining: $30 - 20 = 10$ records.
4. Block 4 (FSM 01): Max occupancy 60 $\rightarrow$ Min free space 40. ($40 \geq 10$).
 - The system checks if it can accommodate the remaining 10 records.

- Actual Space: Block 4 has 35 records. Free space = $100 - 35 = 65$.
- Action: Insert 10 records. New count = $35 + 10 = 45$.
- Remaining: 0 records. Insertion complete.

After insertion, the system performs a full FSM refresh based on the final record counts:

Block	Records (T_1)	Inserted	Final Count	New FSM
1	67	0	67	10
2	95	0	95	11
3	80	+20	100	11
4	35	+10	45	01
5	95	0	95	11
6	95	0	95	11

The final FSM state for blocks 1 – 6 is: 10, 11, 11, 01, 11, 11.

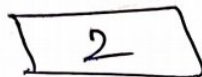
3. $\Rightarrow$ B+ tree constructed

$\Rightarrow$ Start from empty database

$\Rightarrow$ Max. no. of pointers in a node is 4

$\Rightarrow$ B+ tree is constructed with left pointers corresponding to $<$, right pointers corresponding to $\geq$.

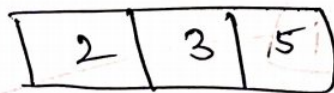
$\Rightarrow$ I2



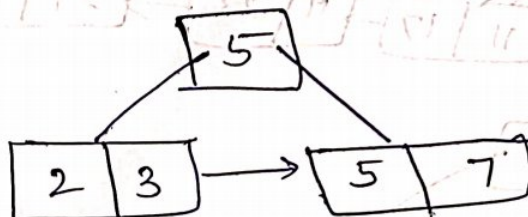
$\Rightarrow$ I3



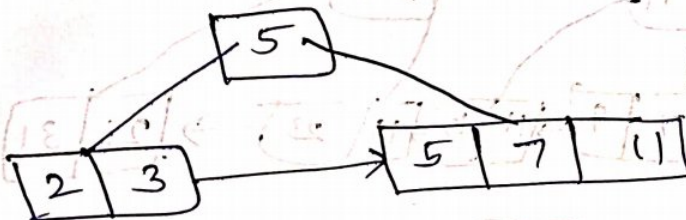
$\Rightarrow$ I5



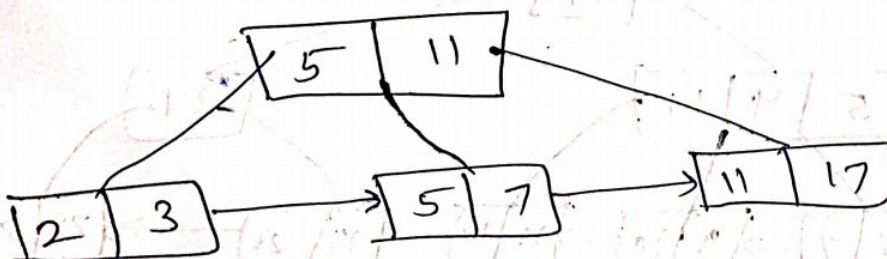
$\Rightarrow$ I7



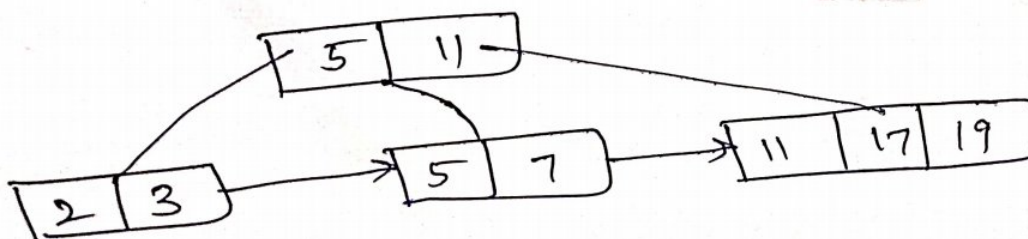
$\Rightarrow$ I11



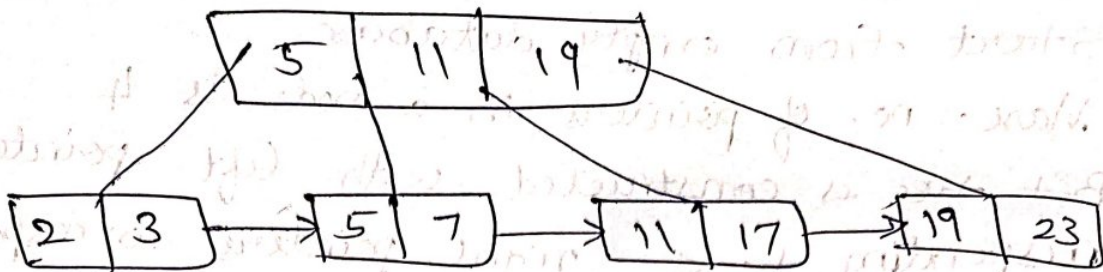
$\Rightarrow$ I17



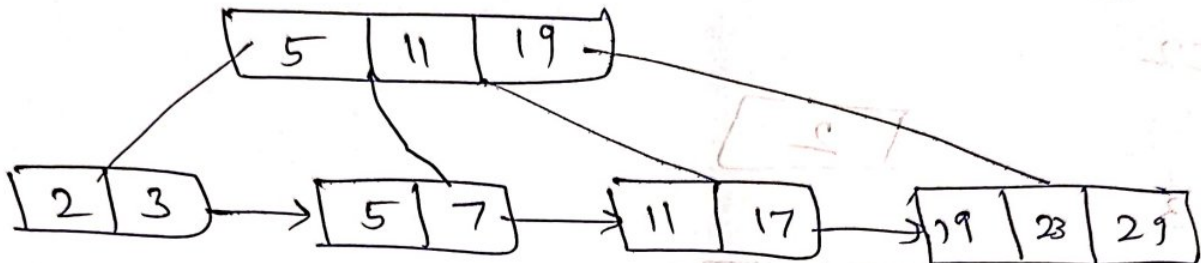
$\Rightarrow$ I19



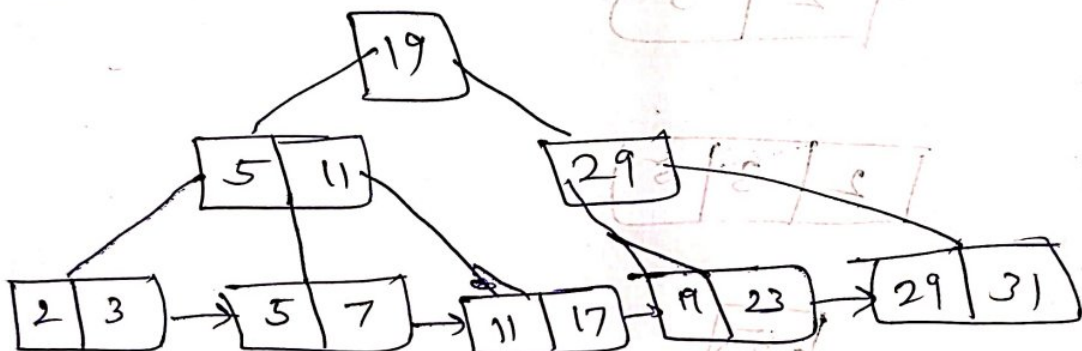
⇒ I 23



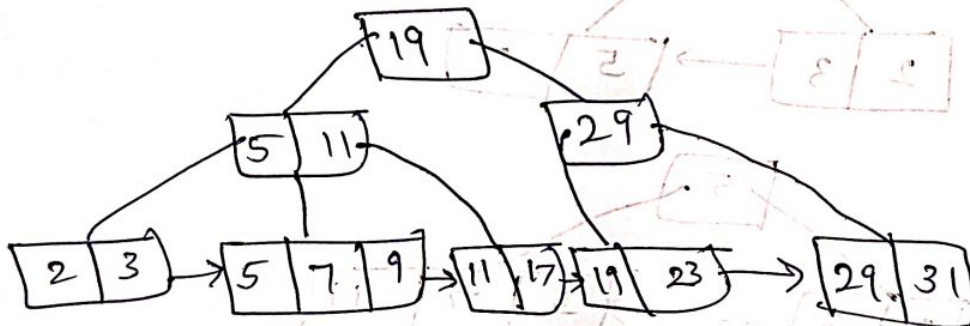
⇒ I 29



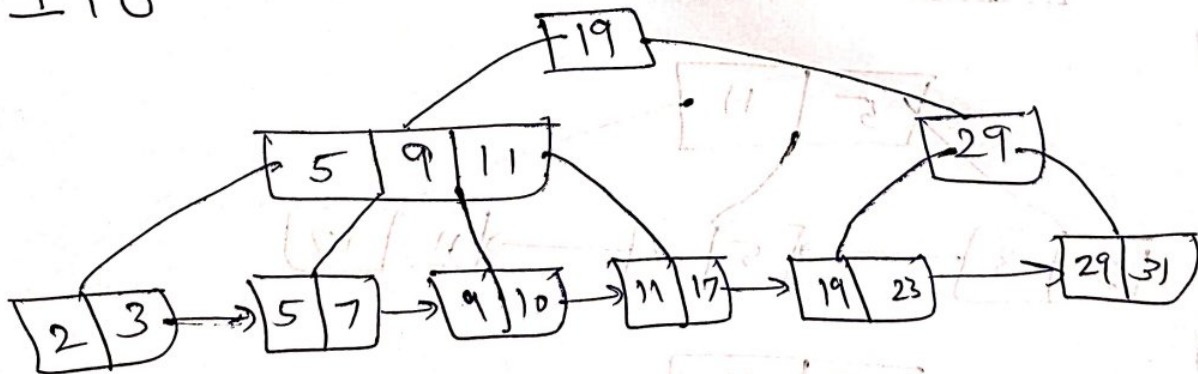
⇒ I 31



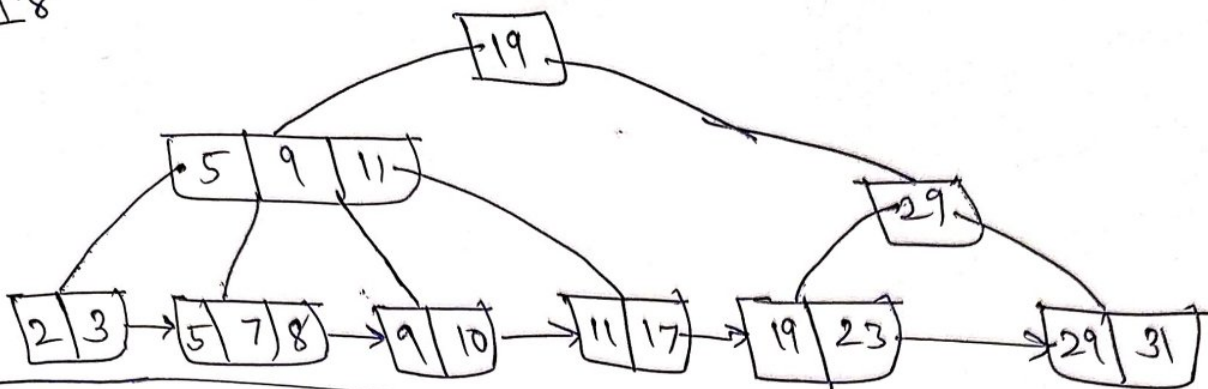
⇒ I 9



⇒ I 10

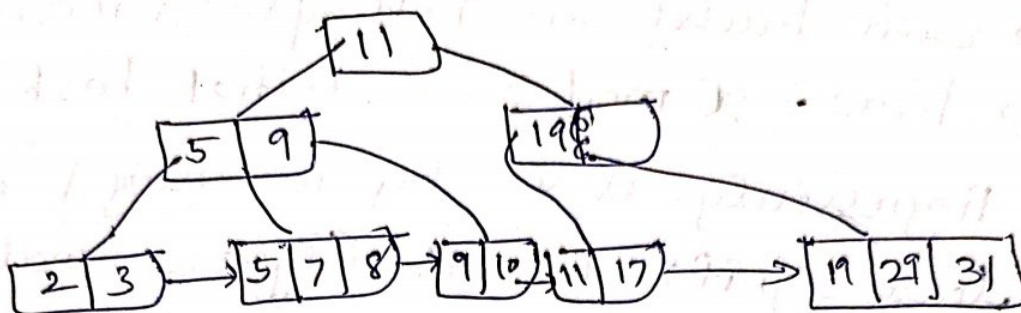


⇒ I8

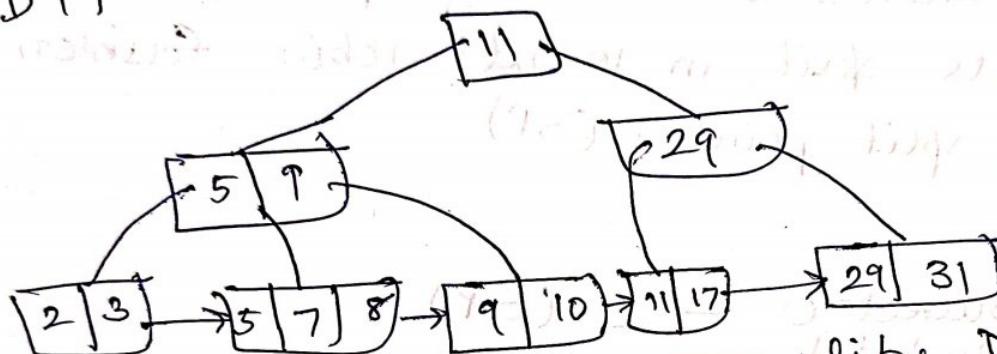


⇒ D23 (handling underflow)

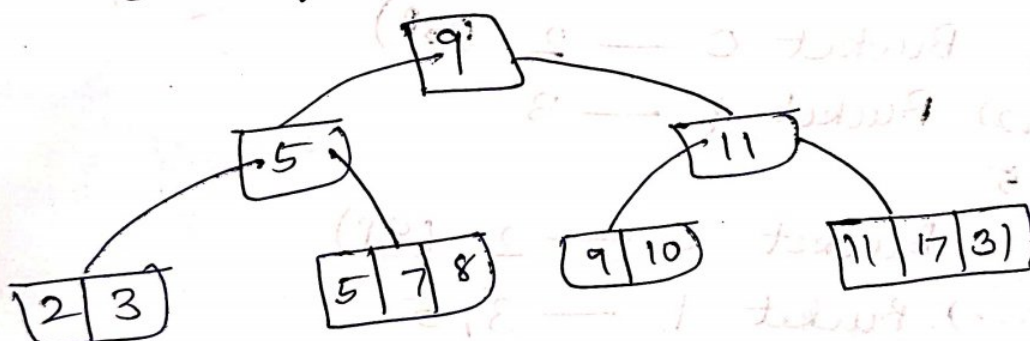
To be done by merging



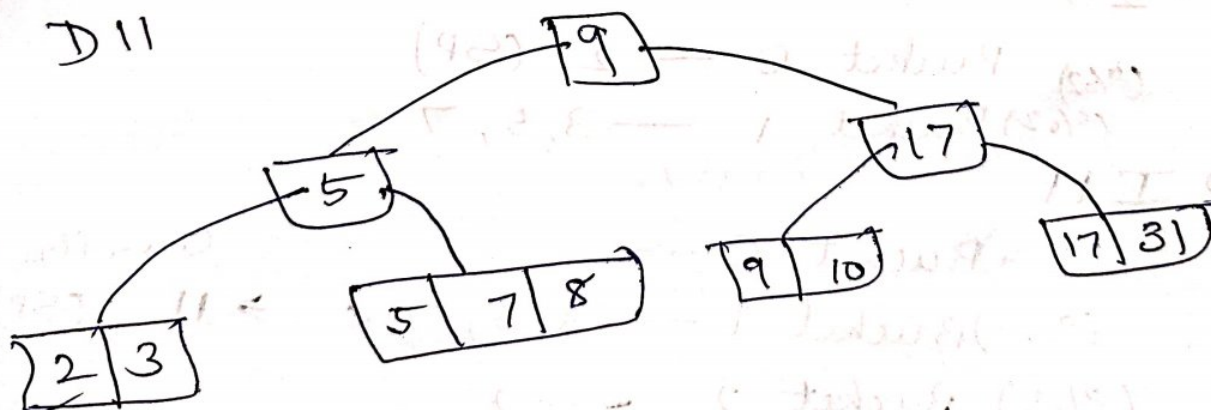
⇒ D19



⇒ D29 (underflow — same like D23)



⇒ D11



4. $\Rightarrow$ Extendible hashing index structure

$\Rightarrow$ Start with empty database

$\Rightarrow$ Hash function $h(x) = x \bmod 8$

$\Rightarrow$ Buckets can hold upto 3 records

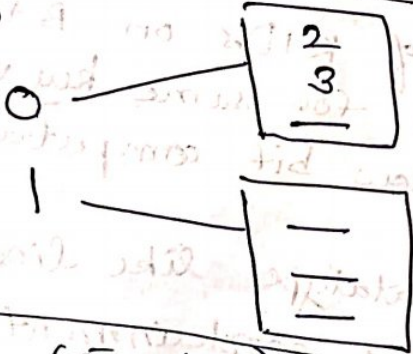
$\Rightarrow$ MSB bits in hash value are used for identifying buckets.

$\Rightarrow I_2$



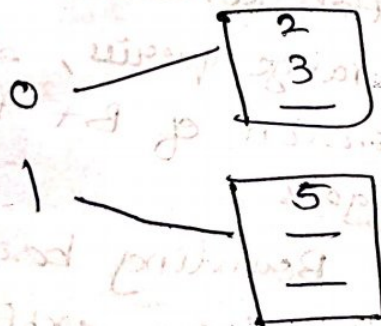
$\Rightarrow I_3$

$011 = 3$

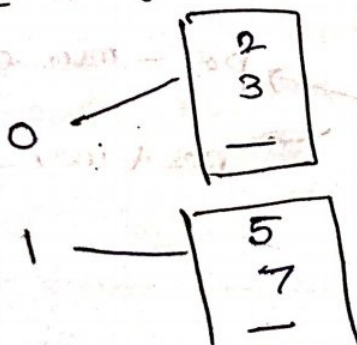


$\Rightarrow I_5$

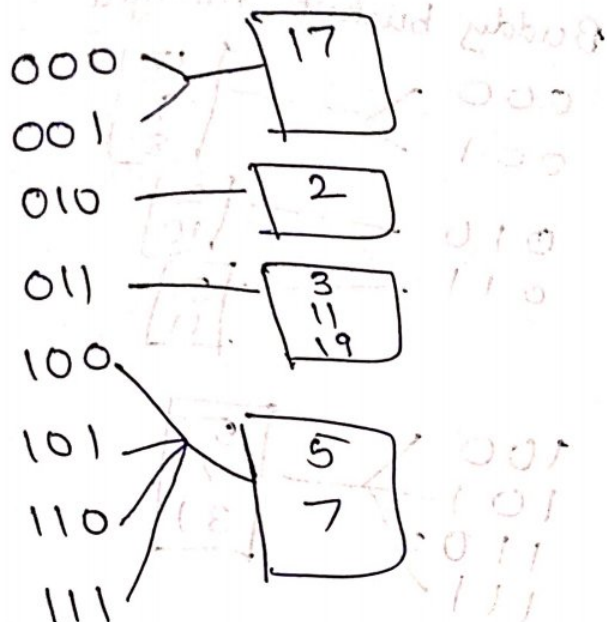
$(5 = 101)$



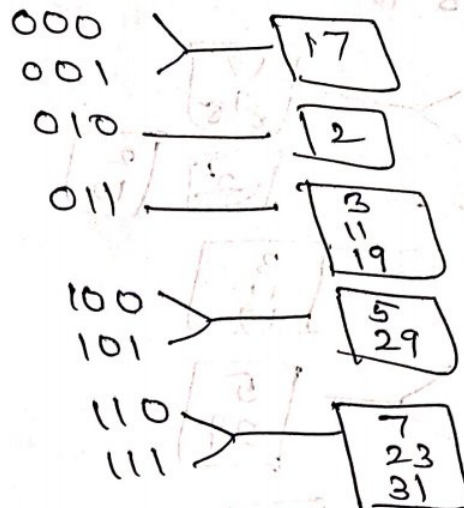
$\Rightarrow I_7$



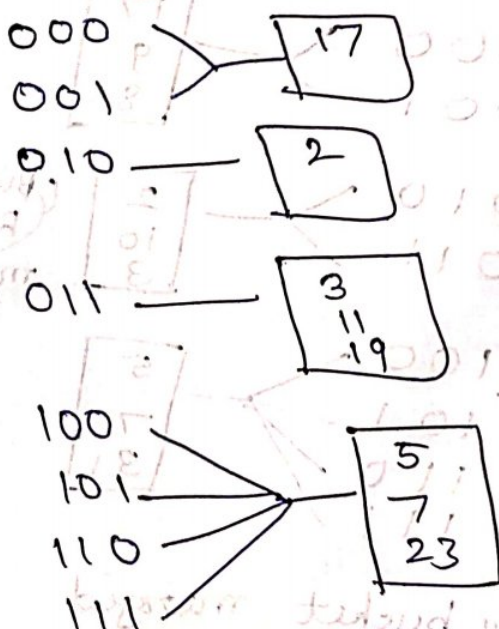
$$\Rightarrow I_{19} (19 \% 8 = 3 = 011)$$



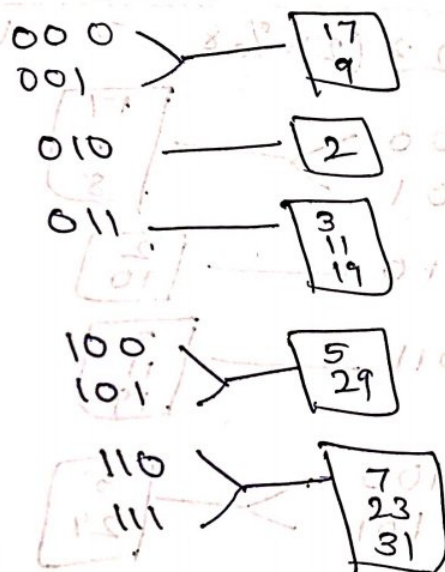
$$\Rightarrow I_{31} (31 \% 8 = 7 = 111)$$



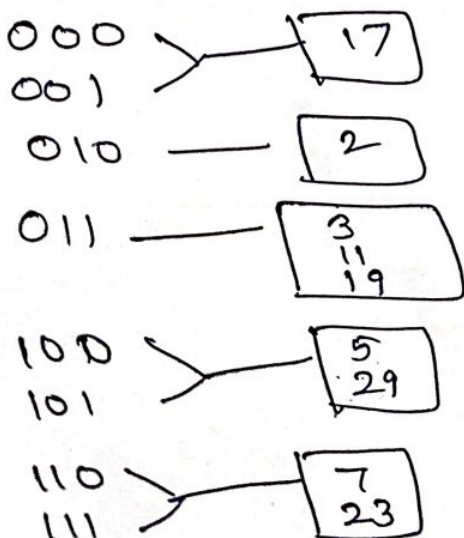
$$\Rightarrow I_{23} (23 \% 8 = 7 = 111)$$



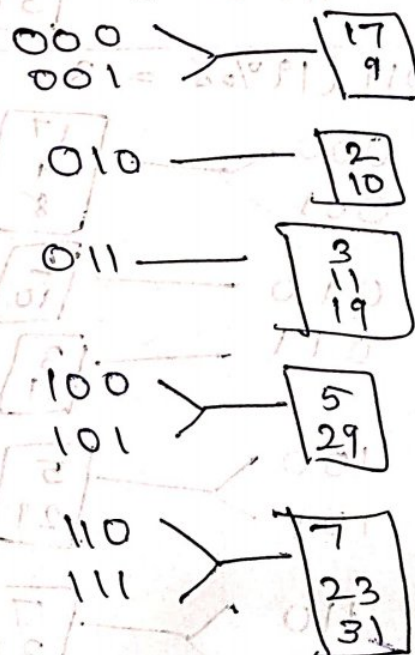
$$\Rightarrow I_9 (9 \% 8 = 1 = 001)$$



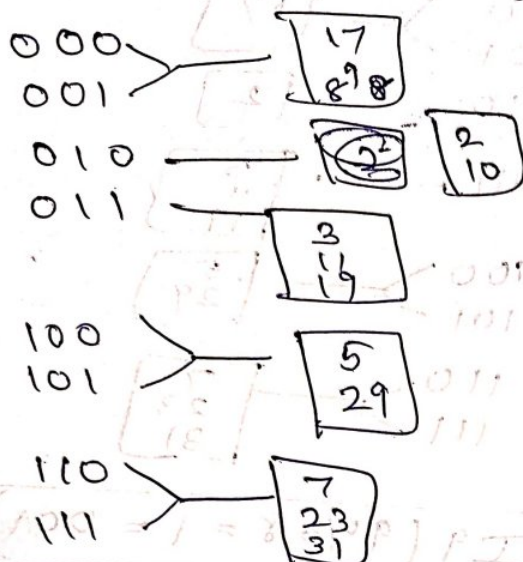
$$\Rightarrow I_{29} (29 \% 8 = 5 = 101)$$



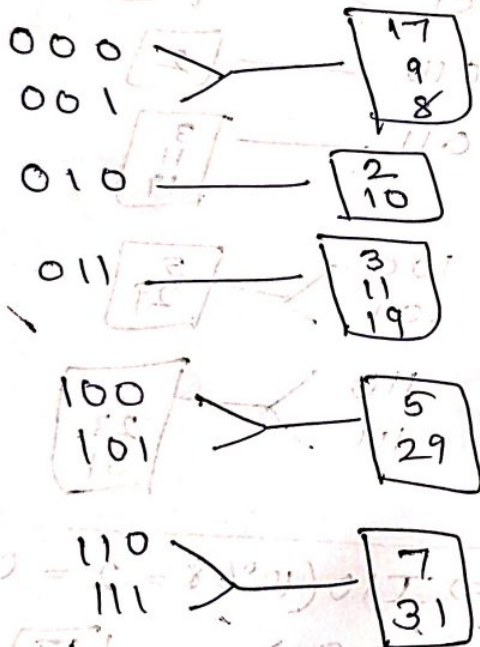
$$\Rightarrow I_{10} (10 \% 8 = 2 = 010)$$



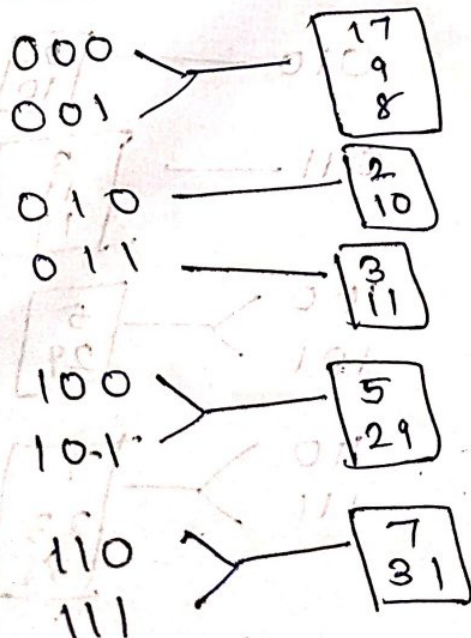
$\Rightarrow I8 (8 \% 8 = 0 = 000)$



$\Rightarrow D23 (23 \% 8 = 7 = 111)$

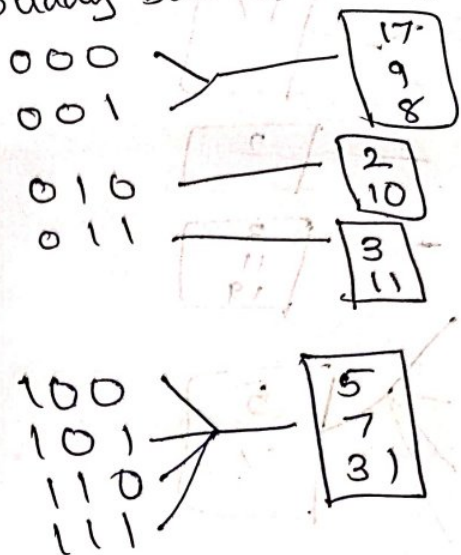


$\Rightarrow D19 (19 \% 8 = 3 = 011)$



$\Rightarrow D29 (29 \% 8 = 5 = 101)$

Buddy bucket merged



$\Rightarrow D11 (11 \% 8 = 3 = 011)$



Buddy bucket merged

5. $\Rightarrow$ Linear hashing index structure

$\Rightarrow$ Start with 2 index blocks

$\Rightarrow$ Each bucket can hold upto 3 records

$\Rightarrow h(x) = x \bmod 2$ — initial hash function

Progressively evolves by revealing 1 additional least significant bit (effective modulo doubles)

for buckets that undergo splitting

$\Rightarrow$ Buckets split in round-robin fashion using split pointer (SP)

$\Rightarrow I_2$

$(\%2)$ Bucket 0 — 2 (SP)

$(\%2)$ Bucket 1 —

$\Rightarrow I_3$

$(\%2)$ Bucket 0 — 2 (SP)

$(\%2)$ Bucket 1 — 3

$\Rightarrow I_5$

$(\%2)$ Bucket 0 — 2 (SP)

$(\%2)$ Bucket 1 — 3, 5

$\Rightarrow I_7$

$(\%2)$ Bucket 0 — 2 (SP)

$(\%2)$ Bucket 1 — 3, 5, 7

$\Rightarrow I_{11}$

$(\%4)$ Bucket 0 —

$(\%2)$ Bucket 1 — 3, 5, 7

$(\%4)$ Bucket 2 — 2

Overflow 1
11 (SP)

$\Rightarrow I_{17}$

$(\%4)$ Bucket 0 —

$(\%2)$ Bucket 1 — 3, 5, 7

$(\%4)$ Bucket 2 — 2

Overflow 2
11, 17 (SP)

⇒ I 19

(%4) Bucket 0 —

(%2) Bucket 1 — 3, 5, 7 ↔ 11, 17, 19 (SP) ^{overflow}

(%4) Bucket 2 — 2

⇒ I 23

(%4) { Bucket 0 — (SP)
Bucket 1 — 5, 17
Bucket 2 — 2
Bucket 3 — 3, 7, 11 ↔ 19, 23 ^{overflow}

⇒ I 29

(%4) { Bucket 0 — (SP)
Bucket 1 — 5, 17, 29
Bucket 2 — 2
Bucket 3 — 3, 7, 11 → 19, 23 ^{overflow}

⇒ I 31

(%4) { Bucket 0 — (SP)
Bucket 1 — 5, 17, 29
Bucket 2 — 2
Bucket 3 — 3, 7, 11 → 19, 23, 31 ^{overflow}

⇒ I 9

(%8) Bucket 0 — ^{overflow}

(%4) { Bucket 1 — 5, 17, 29 → 9 (SP)
Bucket 2 — 2
Bucket 3 — 3, 7, 11 → 19, 23, 31 ^{overflow}

⇒ I 10 (%8) Bucket 4 —

(%8) Bucket 0 —

(%4) { Bucket 1 — 5, 17, 29 → 9 (SP) ^{overflow}
Bucket 2 — 2, 10
Bucket 3 — 3, 7, 11 → 19, 23, 31 ^{overflow}
(%8) Bucket 4 —

⇒ I 8

(%8) Bucket 0 — 8

(%4) { Bucket 1 — 5, 17, 29 → Overflow 1
9 (SP)

{ Bucket 2 — 2, 10

(%8) Bucket 3 — 3, 7, 11 → Overflow 1
19, 23, 31

⇒ D 23

(%8) Bucket 0 — 8

(%4) { Bucket 1 — 5, 17, 29 → Overflow 1
9 (SP)

{ Bucket 2 — 2, 10

(%8) Bucket 3 — 3, 7, 11 → Overflow 1
19, 31

⇒ D 19

(%8) Bucket 0 — 8

(%4) { Bucket 1 — 5, 17, 29 → Overflow 1
9 (SP)

{ Bucket 2 — 2, 10

(%8) Bucket 3 — 3, 7, 11 → Overflow 1
31

⇒ D 29

(%8) Bucket 0 — ~~5, 17, 9~~ 8

(%4) { Bucket 1 — 5, 17, 9 (SP)

{ Bucket 2 — 2, 10

(%8) Bucket 3 — 3, 7, 11 → Overflow 1
31

⇒ D 11

(%8) Bucket 0 — 8

(%4) { Bucket 1 — 5, 17, 9 (SP)

{ Bucket 2 — 2, 10

(%8) Bucket 3 — 3, 7, 31

(%8) Bucket 4 —